
MiniTorch Mentorship Program

Building a Deep Learning Framework from Scratch

By
Oussama BOUSSAHLA

Program Overview

Based on: Cornell Tech's MiniTorch
Duration: **8 Weeks** (2 Months)
Sessions/Week: 2 Live Sessions (16 total)
Format: Theory + Live Coding
Platform: SOAI Algiers — SOAI Labs Program
Assignments: GitHub Classroom (link shared per module)
Modules covered: Module 0 → Module 4 (Networks/CNN)

Project Reference

<https://minitorch.github.io/>
github.com/minitorch/minitorch

Program Philosophy & Learning Arc

MiniTorch is a teaching library from Cornell Tech that lets you **implement every part of a deep learning framework yourself** — from scalar operations and automatic differentiation to tensor storage, broadcasting, GPU kernels, and convolutional networks.

The Core Principle: Always Start from What You Know

Every week follows the same pattern: *understand the concept as a human first, then teach the machine to do it*. Sessions never jump into implementation without first building the mental model.

Learning Arc Across 8 Weeks

Phase	Weeks	What You Are Building
0 — Foundations	1	Tools, skills & ML primer needed to follow the course
1 — Scalar Autodiff	2–4	Differentiation of scalar expressions (Module 0 & 1)
2 — Tensor Engine	5–6	Low-level tensor storage and operations (Module 2)
3 — Efficiency & GPU	7	Parallel ops, CUDA kernels, matmul fusion (Module 3)
4 — Networks & Showcase	8	CNN, pooling, multiclass + final presentations (Module 4)

By the end, every attendee has a **portfolio-grade GitHub project**, a solid grasp of how PyTorch works internally, and the confidence to read real framework source code.

Prerequisites

Mathematics

- **Linear Algebra:** Vectors, matrices, matrix multiplication, transpose, dot product.
- **Calculus:** Single-variable derivatives, partial derivatives, the chain rule.
- **Probability (basic):** Distributions, expectation — helpful but not blocking.

Programming

- **Python 3:** Classes, functions, list comprehensions. Decorators are a bonus.
- **NumPy basics:** Array creation, slicing, broadcasting.
- **Git basics:** Clone, add, commit, push. PRs are a bonus.
- **Command line:** Navigate directories, run scripts, use `pip/conda`.

Optional but Helpful

- Exposure to PyTorch (Tensor, autograd).
- Knowing what a layer, activation function, and loss function are.

Program Rules & Code of Conduct

Attendance & Assignment Policy — Read Carefully

Participation is a **commitment**. The rules below maintain quality and fairness for all attendees.

Attendance

- Each week has **2 live sessions**: one theory session, one live-coding session.
- Attendance at live-coding sessions is **mandatory**.
- **Rule #1**: Missing **3 consecutive live-coding sessions** without prior written justification $\Rightarrow$ **removed from the program**.
- Notify the mentor **before** the session. Justified absences (health, etc) are reviewed individually.

Assignments

- Assignments are distributed and submitted through **GitHub Classroom**. Each module's assignment link will be shared at the start of the relevant week, exactly as done in the official MiniTorch course.
- Accepting the assignment link automatically creates a **personal private repository** for you under the SOAI Algiers GitHub Classroom organisation.
- **Rule #2**: Falling **2 or more assignments behind** the current week $\Rightarrow$ **removed from the program**.
- One late submission (up to 48 h) is tolerated per attendee. After that it counts as missing.
- No submissions via chat or email — GitHub Classroom only.

Academic Integrity & GitHub

- Each attendee works in their own **personal Classroom repository** — do not share repository access with other attendees.
- Commit with meaningful messages (*not* “update” or “fix stuff”).
- Discussing ideas and debugging strategies is **encouraged**. Sharing full solutions is **forbidden**.
- The mentor will review pull requests and leave inline feedback directly inside your Classroom repo.

General Conduct

- Respect all participants and the mentor.
- Ask questions — there are no bad questions.
- Help peers debug; never give them the answer directly.

Session Format

Each week consists of two sessions of approximately **90 minutes**:

Session	Label	Description
Session A	<i>Theory</i>	Concept explained with math, diagrams, and intuition. Includes Q&A. Slides shared after the session.
Session B	<i>Live Coding</i>	Mentor codes the module live; attendees follow in their own Classroom repo. Covers edge cases, tests, and debugging. Mandatory.

Async Support

Between sessions, the mentor is available for questions via the program's Discussions (in Google Classroom) / Discord channel. Code review happens directly on pull requests inside your GitHub Classroom repository.

8-Week Program Plan

Timeline at a Glance

8 weeks × 2 sessions/week = 16 sessions + async GitHub support. Assignment links are released at the start of each relevant week via GitHub Classroom.

Wk	Theme	Phase	Key Topics	Assignment
1	Foundations	Phase 0	Git workflow, functional Python, math warm-up, pytest, project setup, ML primer	Setup
2	Scalar Operators	Phase 1	MiniTorch scaffold, scalar operators, numerical stability, forward functions, property testing	Module 0
3	Scalar Autodiff (fwd)	Phase 1	Computation graphs, chain rule, <code>Variable</code> , <code>ScalarFunction</code>	Module 1
4	Scalar Autodiff (bwd)	Phase 1	Topological sort, <code>backpropagate()</code> , grad checking, test suite	Module 1 (complete)
5	Tensor Internals	Phase 2	Storage, shape, strides, indexing math, <code>to_index</code> , <code>broadcast_index</code>	Module 2-A
6	Tensor Operations	Phase 2	Map/zip/reduce, broadcasting, <code>matrix_multiply</code> , tensor autodiff	Module 2 (complete)
7	Efficiency & GPU	Phase 3	Parallel computation, fusing operations, Numba CUDA kernels, benchmarking	Module 3
8	Networks & Showcase	Phase 4	Convolution, pooling, multiclass (softmax/NLL), final presentations	Module 4 + Demo

Week-by-Week Detail

Week 1 — Foundations & Project Setup

Phase 0

Goal: Every attendee has a working environment, understands the Git workflow, and is comfortable with the Python patterns used throughout MiniTorch. A brief ML primer contextualises why we are building this framework.

Session A — Theory: Tooling, ML Primer & Thinking Like a Framework Builder

- *ML Primer:* what a model is, the training loop at a high level, why we need gradients — just enough to motivate the full 8-week journey.
- *Git workflow:* accepting a Classroom assignment, cloning your repo, branch, commit, push, pull request. Commit message conventions. What the mentor will see in the Classroom dashboard.
- *Functional programming in Python:* higher-order functions, `map`, `filter`, `reduce`; closures; decorators (why MiniTorch uses them extensively).
- *Python tooling:* virtual environments, `pip install -e "[dev]"`, reading `pyproject.toml`, understanding a project scaffold.
- *Testing mindset:* what `pytest` does, how to read a failing test, the red-green-refactor loop. Introduction to **property testing** (Hypothesis) as used extensively in MiniTorch.
- *Math warm-up:* single-variable derivatives, partial derivatives, chain rule — just enough to motivate what comes next.

Session B — Live Coding: Environment + Python Exercises

- Accept the Week 1 GitHub Classroom assignment link (shared in the session). This auto-creates your personal repo — clone it live.
- Create a conda/pip environment, install dependencies, run `pytest tests/`.
- Walk through the folder structure: what each file is for.
- Live exercises: write higher-order functions that mirror the style MiniTorch uses (e.g. a `map` that wraps a scalar function).
- Explore the **visualization tools** bundled with MiniTorch (used from Module 0 onward to track training progress).
- Push the first commit with a proper message. Confirm the mentor can see it in the Classroom dashboard.

Week 1 Deliverable

Classroom repo accepted and cloned, environment runs, `pytest` passes the scaffold tests, and the mentor can see your repository in the Classroom dashboard.

Week 2 — Scalar Operators & Module 0

Phase 1

Goal: Implement and test all scalar operator functions from Module 0 (*Fundamentals*). Understand numerical stability, property testing, and how MiniTorch wraps raw Python math.

Session A — Theory: Operators, Stability & the Module 0 Design

- What is a “scalar operator” in MiniTorch? Pure functions, no state.
- Walk through every operator MiniTorch requires: `mul`, `add`, `neg`, `log`, `exp`, `sigmoid`, `relu`, `lt`, `eq`, `max` ...
- *Numerical stability*: why $\log(1 + \exp(x))$ can overflow, and how to write a stable `log_sigmoid`.
- *EPS guards*: when and why we add a tiny epsilon.
- *Property testing with Hypothesis*: how MiniTorch uses randomised tests to check mathematical laws (commutativity, associativity, etc.) — the `@given` decorator and `strategies`.
- *Modules pattern preview*: the `Module` class that MiniTorch introduces in the Fundamentals assignment for organising parameters — we will build on this structure throughout.
- Reading the test file: what the tests actually check.

Session B — Live Coding: Implement operators.py + Modules

- Accept the **Module 0** GitHub Classroom assignment link (shared at the start of this week).
- Implement every scalar operator one by one, running the relevant test after each implementation.
- Implement the Module 0 `Module` / `Parameter` skeleton (foundation for all later modules).
- Explore the **visualization** dashboard to see operator outputs plotted in real time.
- Debug a failing test live — show the process of reading a traceback.
- Introduce `pytest -k "test_name"` for targeted runs.

Week 2 Deliverable

Module 0 (Fundamentals) tests pass. Code committed and pushed to your Classroom repo.

Week 3 — Autodiff Forward Pass*Phase 1*

Goal: Understand computation graphs and implement the *forward* half of automatic differentiation at the scalar level.

Session A — Theory: What is Autodiff?

- The three ways to differentiate code: numeric, symbolic, automatic.
- *Derivatives review*: limits, single-variable and partial derivatives as used in MiniTorch's *Derivatives* module page.
- *Computation graph*: nodes, edges, how a math expression becomes a DAG.
- **Variable** vs. **ScalarFunction**: the two building blocks.
- *Forward pass*: how values propagate through the graph.
- The chain rule — formal statement and worked example by hand on a simple expression like $f(x) = \sigma(\log x + x^2)$.

Session B — Live Coding: Variable, ScalarFunction, Forward

- Accept the **Module 1** GitHub Classroom assignment link (shared at the start of this week).
- Implement **Variable**: holds a value, tracks its history.
- Implement **ScalarFunction.apply()**: wraps operators to record the graph as it computes.
- Verify forward computation on simple expressions.
- Introduce the **history** concept — what gets stored and why.

Week 3 Deliverable

Variable and **ScalarFunction** implemented. Forward pass produces correct values for scalar expressions. Pushed to Classroom repo.

Week 4 — Autodiff Backward Pass*Phase 1*

Goal: Implement `backpropagate()` and complete Module 1 (*Autodiff*) with all tests green.

Session A — Theory: Reverse-Mode Autodiff

- *Topological sort*: why we need it, how to compute it (DFS + stack).
- *Reverse-mode accumulation*: walk the graph backwards, accumulate derivatives with the chain rule.
- *Gradient accumulation*: why the same variable can receive multiple gradient contributions (fan-in nodes).
- *Numerical gradient checking*: finite differences, when to use it, what a mismatch means.
- Common bugs: forgetting to zero-accumulate, wrong sign, missing nodes.

Session B — Live Coding: `backpropagate()` + Test Suite

- Continue in the Module 1 Classroom repo from Week 3.
- Implement topological sort.
- Implement `backpropagate()`: iterate in reverse order, call each function's `backward()`, accumulate gradients.
- Run the full Module 1 test suite and fix failures live.
- Run numerical gradient checks to verify correctness.

Week 4 Deliverable

Module 1 (Autodiff) tests fully pass. Scalar autodiff is complete end-to-end. Final commit pushed to Classroom repo.

Week 5 — Tensor Internals

Phase 2

Goal: Understand how tensors are stored in memory and implement the low-level indexing machinery that all tensor operations depend on.

Session A — Theory: Memory, Shape, and Strides

- *Flat storage:* how a multi-dimensional tensor lives in a 1-D array.
- *Shape & strides:* what they mean, how to compute them for row-major layout.
- *Index $\leftrightarrow$ position math:* $pos = \sum_i index[i] \times strides[i]$, and the reverse: `to_index`.
- *Broadcasting rules* (step by step): shape alignment, size-1 dimensions, when two shapes are compatible.
- *Views vs. copies:* how strides enable zero-copy reshapes and transposes.

Session B — Live Coding: Indexing Primitives

- Accept the **Module 2** GitHub Classroom assignment link (shared at the start of this week).
- Implement `to_index`: position $\rightarrow$ multi-index.
- Implement `index_to_position`: multi-index $\rightarrow$ flat offset.
- Implement `broadcast_index`: map an output index to an input index under broadcasting rules.
- Verify with hand-worked examples and the Module 2-A test subset.

Week 5 Deliverable

Tensor indexing primitives implemented and tested (Module 2 — indexing subset).
Pushed to Classroom repo.

Week 6 — Tensor Operations & Autodiff*Phase 2*

Goal: Build higher-order tensor operators and their backward passes, including matrix multiply. Complete Module 2 (*Tensors*).

Session A — Theory: Map, Zip, Reduce & Matrix Multiply

- *Higher-order operators:* `map` (element-wise unary), `zip` (element-wise binary with broadcasting), `reduce` (fold along an axis).
- How broadcasting plugs into `zip` via `broadcast_index`.
- *Matrix multiply:* definition, why it is not just a `zip`, the batch dimension.
- *Tensor autodiff:* how gradients flow through `map`, `zip`, `reduce`, and `matmul`. Worked derivations.
- *Tensor auto-grad integration:* connecting Module 2 operators back to the `Variable/backpropagate()` engine from Module 1.

Session B — Live Coding: Operators + Matmul + Tests

- Continue in the Module 2 Classroom repo from Week 5.
- Implement `tensor_map`, `tensor_zip`, `tensor_reduce`.
- Implement `matrix_multiply` (with batch support).
- Implement their backward passes; run gradient checks.
- Run full Module 2 test suite and fix all failures.

Week 6 Deliverable

Module 2 (Tensors) tests fully pass. The tensor engine is complete. Final commit pushed to Classroom repo.

Week 7 — Efficiency: Parallel Computation & GPU*Phase 3*

Goal: Use parallelism and GPU programming to make the tensor engine fast. Implement Module 3 (*Efficiency*): parallel CPU operations, fused matrix multiply, and CUDA kernels via Numba.

Session A — Theory: Parallel Computation & GPU Programming

- *Parallel computation on CPU:* vectorisation, thread pools, Numba's `@jit(parallel=True)`, `prange`.
- *Fusing operations:* why combining map + reduce into a single pass reduces memory bandwidth. The fused matrix multiply algorithm.
- *GPU programming model:* CPU vs. GPU, cores, latency vs. throughput, SIMD thinking.
- CUDA programming concepts: grids, blocks, threads, shared memory.
- *Parallel map on GPU:* each thread handles one element.
- *Parallel reduce on GPU:* tree-based reduction and why it is non-trivial.
- Numba's `@cuda.jit` decorator and memory transfer model.
- When GPU wins and when it does not (memory transfer overhead).

Session B — Live Coding: Parallel Ops + CUDA Kernels + Benchmarks

- Accept the **Module 3** GitHub Classroom assignment link (shared at the start of this week).
- Implement parallel CPU `tensor_map` and `tensor_zip` using Numba `prange`.
- Implement the fused `matrix_multiply` kernel.
- Implement `cuda_map` and `cuda_zip` GPU kernels.
- Benchmark: CPU sequential vs. CPU parallel vs. GPU on large matrices.
- Run Module 3 test suite and fix failures live.

Week 7 Deliverable

Module 3 (Efficiency) tests pass. Parallel CPU and CUDA GPU kernels implemented. Benchmark results recorded. Pushed to Classroom repo.

Week 8 — Foundational Deep Networks & Final Showcase*Phase 4*

Goal: Use the full MiniTorch stack to build convolutional and multiclass networks. Complete Module 4 (*Networks*) and present your finished framework.

Session A — Theory: Convolution, Pooling & Multiclass Classification

- *Convolution:* 1-D and 2-D convolution as a sliding dot product, the kernel / filter concept, valid vs. same padding.
- *Backward through convolution:* gradient w.r.t. input and kernel — derived from the forward definition.
- *Pooling:* max-pooling and average-pooling, how they reduce spatial dimensions, backward pass for max-pooling.
- *Multiclass classification:* softmax function, log-softmax for numerical stability, negative log-likelihood (NLL) loss.
- *Putting it together:* a full CNN pipeline — conv $\rightarrow$ pool $\rightarrow$ flatten $\rightarrow$ linear $\rightarrow$ softmax.

Session B — Live Coding + Final Showcase

- Accept the **Module 4** GitHub Classroom assignment link (shared at the start of this week).
- Implement `conv1d` and `conv2d` (and their backward passes).
- Implement `maxpool2d` and `avgpool2d`.
- Implement `softmax` and `logsoftmax`; wire up NLL loss.
- Train a small CNN on a digit dataset using your own framework.
- **Final Showcase (20 min each):** every attendee presents their Classroom repo, runs their trained model, and explains one thing they found hard and how they solved it.

Week 8 Deliverable

Module 4 (Networks) attempted. A trained convolutional network built entirely on your own MiniTorch stack. Final Classroom repo is clean, documented, and portfolio-ready.

GitHub Classroom Setup

Complete Before the First Session

All assignments are distributed via **GitHub Classroom**. Each module's invitation link will be shared at the beginning of the relevant week — just like in the official MiniTorch course. Do **not** fork the MiniTorch repository manually; always use the Classroom link.

1. Create a free account at github.com if you do not already have one.
2. When a Classroom assignment link is shared, click it and sign in with your GitHub account. Accept the assignment — GitHub will automatically create a **private repository** for you under the MiniTorch-with-BSHLoussama Classroom organisation.
3. Clone your new Classroom repository:

```
eg.git clone https://github.com/org-id/module-N-YOUR_USERNAME.git
```
4. Create a Python 3.8+ environment and install:

```
pip install -e ".[dev]"
```
5. Verify: `pytest tests/` — most tests will be skipped; that is normal.
6. After each session, commit & push with a descriptive message, e.g.:

```
git commit -m "Week 3: implement Variable and ScalarFunction apply"
```
7. The mentor monitors submissions directly from the Classroom dashboard — no need to share your repo URL separately.

Assignment Link Release Schedule

Links are released **one module at a time** at the start of each relevant week. Do not wait for all links at once — they will not all be available from day one. Watch the Discord / Discussions channel for announcements.

Quick Reference Card

Rules at a Glance

1. **3 consecutive absences** from live-coding sessions without justification ⇒ **removed**.
2. **2 or more assignments behind** the current week ⇒ **removed**.
3. Push to your **GitHub Classroom repo** after every session.
4. **Discuss ideas**; never share full solutions.
5. Communicate absences **before** the session.
6. Assignment links are released **one at a time** — watch Discord for announcements.

See you there!!